

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun

Abstract

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers — $8\times$ deeper than VGG nets but still having lower complexity. An ensemble of these residual nets achieves 3.57% error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers. The depth of representations is of central importance for many visual recognition tasks. Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset. Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions, where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation.

1 Introduction

Deep convolutional neural networks have driven a series of breakthroughs in image classification [18]. Depth is of crucial importance: leading results on the challenging ImageNet dataset exploit very deep models, and the features they learn span low/mid/high-level abstractions that many other visual recognition tasks have benefited from [47]. Driven by this significance, a natural question arises: *is learning better networks as easy as stacking more layers?*

An obstacle to answering this question is the notorious problem of vanishing / exploding gradients [1], which hampers convergence from the beginning. This problem, however, has been largely addressed by normalized initialization [6, 19, 34] and intermediate normalization layers, which enable networks with tens of layers to start converging for stochastic gradient descent (SGD) with backpropagation. When deeper networks are able to start converging, a degradation problem has been exposed [8]: with the network depth increasing, accuracy

gets saturated and then degrades rapidly. Unexpectedly, such degradation is *not* caused by overfitting, and adding more layers to a suitably deep model leads to *higher training error*. On CIFAR-10, a 56-layer plain network reaches higher training and test error than a 20-layer one; the same phenomenon appears on ImageNet, where a 34-layer plain network obtains higher validation error (28.54% top-1) than its 18-layer counterpart (27.94% top-1).

The degradation of training accuracy indicates that not all systems are similarly easy to optimize. Consider a shallower architecture and its deeper counterpart that adds more layers onto it. There exists a solution by *construction* to the deeper model: the added layers are identity mappings, and the other layers are copied from the learned shallower model. The existence of this constructed solution indicates that a deeper model should produce no higher training error than its shallower counterpart. Yet experiments show that our current solvers are unable to find solutions that are comparably good or better than the constructed solution. Because the plain networks here use batch normalization, which ensures forward signals have non-zero variances, and the backward propagated gradients are verified to exhibit healthy norms, we *argue* that the difficulty is one of *optimization* and that vanishing gradients are an *unlikely* cause; we do not, however, formally prove this, and we conjecture that the deep plain networks may have exponentially low convergence rates.

In this paper, we address the degradation problem by introducing a *deep residual learning* framework. Instead of hoping each few stacked layers directly fit a desired underlying mapping, we explicitly let these layers fit a residual mapping. Formally, denoting the desired underlying mapping as $\mathcal{H}(\mathbf{x})$, we let the stacked nonlinear layers fit another mapping of $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original mapping is recast into $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. We hypothesize that it is easier to optimize the residual mapping than to optimize the original, unreferenced mapping. To the extreme, if an identity mapping were optimal, it would be easier to push the residual to zero than to fit an identity mapping by a stack of nonlinear layers. The formulation of $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ can be realized by feedforward neural networks with *shortcut connections* that simply perform identity mapping; these connections add neither extra parameters nor computational complexity, and the entire network can still be trained end-to-end by SGD with backpropagation.

We present comprehensive experiments on ImageNet to

show the degradation problem and evaluate our method. We show that the residual networks are easy to optimize, whereas the counterpart plain networks exhibit higher training error when depth increases; and that the residual networks can easily enjoy accuracy gains from greatly increased depth, producing results substantially better than previous networks. Similar phenomena are also shown on the CIFAR-10 set, where successful models at over 100 layers are presented and a model with over 1000 layers is explored. On the ImageNet classification dataset, we obtain excellent results by extremely deep residual nets: a 152-layer residual net is the deepest network ever presented on ImageNet while still having lower complexity than VGG nets, and an ensemble has 3.57% top-5 error on the test set, winning the 1st place in the ILSVRC 2015 classification competition. The extremely deep representations also have excellent generalization performance on other recognition tasks: replacing the VGG-16 backbone with a 101-layer residual net in a baseline detection system yields a 28% relative improvement on the COCO object detection dataset, indicating that the depth of representations is of central importance for many visual recognition tasks.

2 Related Work

Residual representations. The idea of learning with respect to residuals has a long history in vision and numerical computation. In image recognition, encodings such as VLAD and the Fisher Vector represent images by residual vectors with respect to a dictionary, and encoding residuals has been shown to be more effective than encoding original vectors for shallow retrieval and classification pipelines [15, 14, 27, 45]. In low-level vision and graphics, solving partial differential equations through multigrid methods or hierarchical basis preconditioning reformulates a system in terms of residual variables, and solvers that are aware of this residual nature converge far faster than ones that are not [42, 43, 44]. These results suggest that a good reformulation or preconditioning can simplify optimization. We extend this principle from hand-designed encodings and solvers to the internal layers of deep networks, reformulating each block as a residual function so the optimizer operates on residuals rather than unreferenced mappings.

Shortcut connections. Practices and theories that lead to shortcut connections have been studied for a long time [2]. Early multi-layer perceptrons were trained with a linear layer connecting the input to the output [31, 46], and later work centered layer responses, gradients, and propagated errors through shortcut connections [28, 35, 36]. Inception layers are also composed of a shortcut branch and a few deeper branches [41]. Concurrent with our work, “highway networks” present shortcut connections with gating functions [39, 40, 12]. These gates are data-dependent and have parameters, in

contrast to our parameter-free identity shortcuts. When a gated shortcut is “closed” (approaching zero), the layers in highway networks represent non-residual functions; by contrast, our formulation always learns residual functions, our identity shortcuts are never closed, and all information is always passed through, with additional residual functions to be learned. In addition, highway networks have not demonstrated accuracy gains with extremely increased depth (e.g., over 100 layers).

Deep network baselines and enabling techniques. VGG nets establish the design philosophy of stacking small 3×3 convolutions to great depth, and serve as the cleanest plain network baseline against which residual learning is measured [38]. We adopt VGG’s design rules but operate at far greater depth and lower complexity. Batch normalization is applied after each convolution and before activation in both our plain networks and residual nets; it ensures forward signals maintain non-zero variances and lets us isolate the optimization-difficulty hypothesis for the degradation problem [13]. All convolutional weights, in both plain network and residual variants, are initialized with the MSRA scheme for rectifier networks [10]. Auxiliary-classifier and dropout-based regularizers used by earlier deep models [21, 11] are deliberately omitted so that the depth ablation remains clean. We further compare our single-model and ensemble ImageNet results against GoogLeNet, PReLU-net, and BN-inception as strong contemporary reference points [41, 10, 13].

Transfer to detection. To study whether deeper representations help beyond classification, we build on the Faster R-CNN detection framework, which couples a region proposal network with a Fast R-CNN detector [29, 4, 5]. Because our 101-layer residual net lacks the hidden fully-connected layers that the VGG backbone provides, we adopt the strategy of treating the convolutional feature maps as shared full-image features and the final convolutional stage as the per-region equivalent of the fully-connected layers [30, 9]. Holding the detection hyperparameters fixed and swapping only the backbone isolates the contribution of the learned representation. We evaluate on the standard PASCAL VOC and COCO detection benchmarks [3, 23, 33].

3 Deep Residual Learning

3.1 Residual Learning

Let us consider $\mathcal{H}(\mathbf{x})$ as an underlying mapping to be fit by a few stacked layers, with $\mathbf{x}$ denoting the inputs to the first of these layers. If one hypothesizes that multiple non-linear layers can asymptotically approximate complicated functions, then it is equivalent to hypothesize that they can asymptotically approximate the residual function, i.e., $\mathcal{H}(\mathbf{x}) - \mathbf{x}$ (assuming that the input and output are of the same dimensions). So rather than expecting stacked layers to approximate $\mathcal{H}(\mathbf{x})$, we explicitly let these layers

approximate a residual function $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original function thus becomes $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. Although both forms should be able to asymptotically approximate the desired functions, as hypothesized, the ease of learning might be different. This reformulation is motivated by the counterintuitive degradation problem: if the added layers can be constructed as identity mappings, a deeper model should have training error no greater than its shallower counterpart. The degradation problem suggests that solvers might have difficulties in approximating identity mappings by multiple nonlinear layers. With the residual learning reformulation, if identity mappings are optimal, the solvers may simply drive the weights of the multiple nonlinear layers toward zero to approach identity mappings. We note that the hypothesis that stacked nonlinear layers can asymptotically approximate the residual function remains an open question [25].

3.2 Identity Mapping by Shortcuts

We adopt residual learning to every few stacked layers. A building block is defined as

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (1)$$

Here $\mathbf{x}$ and $\mathbf{y}$ are the input and output vectors of the layers considered. The function $\mathcal{F}(\mathbf{x}, \{W_i\})$ represents the residual mapping to be learned. For the two-layer basic block illustrated in Fig. 1, $\mathcal{F} = W_2 \sigma(W_1 \mathbf{x})$, in which σ denotes ReLU and the biases are omitted for simplifying notations. The operation $\mathcal{F} + \mathbf{x}$ is performed by a shortcut connection and element-wise addition, and we adopt the second nonlinearity after the addition, i.e., $\sigma(\mathbf{y})$. The shortcut connections in Eq. (1) introduce neither extra parameter nor computation complexity, which is important for our comparisons between plain networks and residual nets that have the same number of parameters, depth, width, and computational cost.

The dimensions of $\mathbf{x}$ and $\mathcal{F}$ must be equal in Eq. (1). When this is not the case — for example, when changing the input/output channels at a stage boundary — we perform a linear projection W_s by the shortcut connection to match the dimensions:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s \mathbf{x}. \quad (2)$$

The form of the residual function $\mathcal{F}$ is flexible, but a function with a single layer is observed to provide no advantage, behaving similarly to a linear layer; we therefore use a residual function $\mathcal{F}$ with two or more layers.

3.3 Network Architectures

We have tested various plain networks and residual nets, and have observed consistent phenomena. The plain network baselines are mainly inspired by the philosophy of VGG nets. The convolutional layers mostly have 3×3 filters and follow two simple design rules: (i) for the same

output feature map size, the layers have the same number of filters; and (ii) if the feature map size is halved, the number of filters is doubled so as to preserve the time complexity per layer. We perform downsampling directly by convolutional layers that have a stride of 2. The network ends with a global average pooling layer and a 1000-way fully-connected layer with softmax. It is worth noticing that our model has fewer filters and lower complexity than VGG nets: our 34-layer plain network has 3.6 billion FLOPs, which is only 18% of VGG-19 (19.6 billion FLOPs).

Based on the plain network, we insert shortcut connections which turn the network into its counterpart residual version. The identity shortcuts of Eq. (1) can be used directly when the input and output are of the same dimensions. When the dimensions increase, we consider two options: (A) the shortcut still performs identity mapping, with extra zero entries padded for increasing dimensions, introducing no extra parameter; or (B) the projection shortcut of Eq. (2) is used to match dimensions, done by 1×1 convolutions. For both options, when the shortcuts go across feature maps of two sizes, they are performed with a stride of 2. Table 1 details the five-stage skeleton shared by all depths, with the per-stage block counts and overall complexity.

3.4 Deeper Bottleneck Architectures

For deeper nets, we modify the building block as a *bottleneck* design out of concern for the training time that we can afford. For each residual function $\mathcal{F}$, we use a stack of three layers instead of two. The three layers are 1×1 , 3×3 , and 1×1 convolutions, where the 1×1 layers are responsible for reducing and then restoring dimensions, leaving the 3×3 layer a bottleneck with smaller input/output dimensions. This design has a time complexity similar to that of the two-layer block. The parameter-free identity shortcuts are particularly important for the bottleneck architectures: if the identity shortcut is replaced with a projection, one can show that the time complexity and model size are doubled, as the shortcut is connected to the two high-dimensional ends. So identity shortcuts lead to more efficient models for the bottleneck designs. We construct 50-, 101-, and 152-layer residual nets by using this bottleneck block, with the per-stage block counts in Table 1. Notably, the 152-layer residual net has 11.3 billion FLOPs, which is still of lower complexity than VGG-16/19 nets (15.3/19.6 billion FLOPs).

3.5 Implementation

For each residual block we follow the forward computation summarized in Algorithm 1. The shortcut branch is selected once at construction time according to whether the block changes dimensions and which shortcut option is in effect; identity shortcuts dominate, with projections used only at the rare stage boundaries. Our training

Table 1: Architectures for ImageNet. Building blocks are shown in brackets, with the numbers of blocks stacked. Downsampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2. *Source: Table 1 (evidence/tables/table1_imagenet_architectures.md); exact FLOPs and block counts.*

layer name	output size	18-layer	34-layer	50-layer	101/152-layer
conv1	112×112	7×7 , 64, stride 2			
conv2_x	56×56	3×3 max pool, stride 2			
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times \{4, 8\}$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times \{23, 36\}$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax			
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	$7.6 \times 10^9 / 11.3 \times 10^9$

Algorithm 1 Forward pass of one residual block.

Require: feature $\mathbf{x}$; residual function $\mathcal{F}$ (basic or bottleneck); shortcut option; downsample flag

- 1: $\text{out} \leftarrow \mathcal{F}(\mathbf{x})$ $\triangleright$ conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ conv $\rightarrow$ BN (or $1 \times 1, 3 \times 3, 1 \times 1$)
- 2: **if** $\dim(\text{out}) = \dim(\mathbf{x})$ **and** not downsample **then**
- 3: $\text{id} \leftarrow \mathbf{x}$ $\triangleright$ parameter-free identity
- 4: **else**
- 5: **if** option = A **then**
- 6: $\text{id} \leftarrow \text{ZEROPADCHANNELS}(\text{STRIDE2}(\mathbf{x}))$
- 7: **else**
- 8: $\text{id} \leftarrow \text{PROJECTION}(\mathbf{x}) \triangleright 1 \times 1$ conv (+stride 2)
- 9: **end if**
- 10: **end if**
- 11: $\text{out} \leftarrow \text{out} + \text{id}$ $\triangleright$ element-wise add
- 12: **return** $\text{ReLU}(\text{out})$

recipe follows the established practice for deep networks on ImageNet: batch normalization is applied right after each convolution and before activation, weights are initialized with the MSRA scheme, and the networks are trained from scratch by SGD. Exact optimizer settings, augmentation, and test-time protocols are deferred to the Appendix so that the body remains at the level of the architectural contribution.

4 Experiments

4.1 ImageNet Classification

We evaluate our method on the ImageNet 2012 classification dataset that consists of 1000 classes. The models are trained on the 1.28 million training images and evaluated on the 50,000 validation images; we also obtain a final result on the 100,000 test images, reported by the test server. We evaluate both top-1 and top-5 error rates.

Table 2: Top-1 error (% , 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. *Source: Table 2 (table2_imagenet_plain_vs_residual.md); exact.*

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Plain networks vs. residual networks. We first evaluate 18-layer and 34-layer plain networks and their residual counterparts under an identical training pipeline; the residual nets use identity shortcuts with zero-padding for increasing dimensions (Option A), so they have *no extra parameter* compared to the plain network counterparts. Table 2 reports the top-1 validation error with 10-crop testing. The 34-layer plain network has higher validation error (28.54%) than the 18-layer plain network (27.94%), exhibiting the degradation problem: the deeper plain network is worse despite its strictly larger solution space. The underlying training-error analysis reports that the 34-layer plain network’s training error remains higher than the 18-layer one’s throughout the optimization, indicating that this is an optimization difficulty rather than a final-step effect; per-iteration curves are not reproduced here (see Fig. 2 and the limitation in Section 5). The situation is reversed for the residual nets: the 34-layer ResNet attains 25.03% top-1 error, which is 3.51 points better than the 34-layer plain network and 2.85 points better than the 18-layer ResNet (27.88%). This comparison verifies that residual learning removes the degradation problem and lets accuracy grow with depth at matched parameter count.

Identity vs. projection shortcuts. We next examine projection shortcuts. Table 3 isolates the three shortcut options on the 34-layer net against the plain network base-

Table 3: ResNet-34 shortcut-option ablation on ImageNet validation (% , 10-crop), against the plain-34 baseline. *Source: derived subset of Table 3 (derived_from_table3_shortcut_options.md); exact.*

model	shortcut	top-1	top-5	Δ top-1
plain-34	none	28.54	10.02	—
ResNet-34 A	identity + zero-pad	25.03	7.76	−3.51
ResNet-34 B	proj. on dim. change	24.52	7.46	−4.02
ResNet-34 C	proj. everywhere	24.19	7.40	−4.35

line: (A) zero-padding identity shortcuts, parameter-free; (B) projection shortcuts only for increasing dimensions, with all other shortcuts being identity; and (C) projection shortcuts for all blocks. All three options are considerably better than the plain network (−3.51 to −4.35 top-1 points). Among the three, B is slightly better than A; we argue this is because the zero-padded dimensions in A indeed have no residual learning. C is marginally better than B, which we attribute to the extra parameters introduced by the thirteen projection shortcuts. The small differences among A/B/C (≤ 0.65 top-1 points) indicate that projection shortcuts are not essential for addressing the degradation problem. We therefore do not use option C in the remainder of the paper, to reduce memory/time complexity and model sizes; identity shortcuts are particularly important for not increasing the complexity of the bottleneck architectures introduced next.

Going deeper with bottleneck blocks. Using the bottleneck design (Section 3.4), we construct 50-, 101-, and 152-layer residual nets with option B shortcuts and train them with the same recipe. Table 4 reports the full 10-crop validation errors against contemporary plain network and competitor baselines. The top-1 error decreases monotonically with depth, from 24.52% (ResNet-34 B) to 22.85%, 21.75%, and 21.43% for ResNet-50, ResNet-101, and ResNet-152 respectively, with the matching top-5 error falling from 7.46% to 5.71%. Because residual blocks add no parameters relative to their plain counterparts, this gain within the studied depths is attributable to depth rather than added width. Figure 3 plots the trend. The deeper bottleneck nets also remain efficient: ResNet-152 has 11.3×10^9 FLOPs (Table 1), still below VGG-16/19.

Single-model and ensemble results. With multi-scale fully-convolutional testing, the single-model ResNet-152 reaches 19.38% top-1 / 4.49% top-5 error on the validation set, the best single-model result among the entries in Table 5 and below the prior PReLU-net and BN-inception single models. Finally, an ensemble of six models of different depth attains **3.57%** top-5 error on the ImageNet *test* set (Table 6), which won the 1st place in the ILSVRC 2015 classification competition.

Table 4: Error rates (% , 10-crop testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions. *Source: Table 3 (table3_imagenet_validation_full.md); exact.*

model	top-1 err.	top-5 err.
VGG-16 [38]	28.07	9.33
GoogLeNet [41]	—	9.15
PReLU-net [10]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 5: Single-model results on the ImageNet validation set (except † reported on the test set). *Source: Table 4 (table4_imagenet_singlemodel.md); exact, caption qualifier preserved.*

method	top-1 err.	top-5 err.
VGG [38] (ILSVRC’14)	—	8.43†
GoogLeNet [41] (ILSVRC’14)	—	7.89
VGG [38] (v5)	24.4	7.1
PReLU-net [10]	21.59	5.71
BN-inception [13]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

4.2 CIFAR-10 and Analysis

We conducted more studies on the CIFAR-10 dataset, which consists of 50,000 training and 10,000 testing images in 10 classes. The CIFAR networks use a $6n+2$ stacked-layer skeleton with $\{16, 32, 64\}$ filters across the three $\{32, 16, 8\}$ feature-map sizes, and option A identity shortcuts everywhere, so the residual nets have exactly the same depth, width, and number of parameters as the plain counterparts. Table 7 reports the test error against baselines. The residual nets improve as depth grows from 20 to 110 layers, with test error decreasing $8.75 \rightarrow 7.51 \rightarrow 7.17 \rightarrow 6.97 \rightarrow 6.43\%$; the 110-layer ResNet (1.7M parameters) reaches 6.43% (best; mean $\pm$ std 6.61 ± 0.16 over 5 runs), competitive with far larger prior models. Figure 4 plots the trend, including the 1202-layer point discussed below. The corresponding plain networks, by contrast, suffer from the degradation problem as depth increases, and the 110-layer plain network reaches an error higher than 60% and is not displayed.

Warmup for the 110-layer net. For the 110-layer

Table 6: Error rates (%) of ensembles. The top-5 error is on the test set of ImageNet and reported by the test server. *Source: Table 5 (table5_imagenet_ensembles.md); exact.*

method	top-5 err. (test)
VGG [38] (ILSVRC'14)	7.32
GoogLeNet [41] (ILSVRC'14)	6.66
VGG [38] (v5)	6.8
PReLU-net [10]	4.94
BN-inception [13]	4.82
ResNet (ILSVRC'15)	3.57

Table 7: Classification error on the CIFAR-10 test set. All methods are with data augmentation. For ResNet-110, we run it 5 times and show “best (mean $\pm$ std)”. *Source: Table 6 (table6_cifar10.md); exact, caption qualifier preserved.*

method	#layers	#params	error (%)
Maxout [7]	—	—	9.38
NIN [22]	—	—	8.81
DSN [21]	—	—	8.22
FitNet [32]	19	2.5M	8.39
Highway [39, 40]	19	2.3M	7.54 (7.72 $\pm$ 0.16)
Highway [39, 40]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61 $\pm$ 0.16)
ResNet	1202	19.4M	7.93

ResNet, an initial learning rate of 0.1 is slightly too large to start converging. We therefore warm up the training with a learning rate of 0.01 for the first ~ 400 iterations until the training error drops below $\sim 80\%$, and then restore the learning rate to 0.1 to continue. The remaining schedule is unchanged. We note this warmup is a stability heuristic specific to this depth rather than a fundamental requirement of residual learning.

Exploring over 1000 layers. We further explore an aggressively deep model of 1202 layers (19.4M parameters). Our method shows no optimization difficulty at this depth, and the 1202-layer net achieves a training error below 0.1%. Its test error, however, is 7.93% — worse than the 110-layer net (Table 7, Fig. 4). We argue this is because of overfitting: the 1202-layer network is unnecessarily large for this small dataset, and we did not use strong regularization such as maxout or dropout in this work. The result is reported as a study of optimization behavior at extreme depth, not as a recommended operating point.

Table 8: Object detection mAP (%) on the PASCAL VOC 2007/2012 test sets using baseline Faster R-CNN. *Source: Table 7 (table7_pascal_voc_detection.md); exact.*

	07+12 (VOC 07 test)	07++12 (VOC 12 test)
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

Table 9: Object detection mAP (%) on the COCO validation set using baseline Faster R-CNN. *Source: Table 8 (table8_coco_detection.md); exact.*

	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

4.3 Object Detection on PASCAL and COCO

Our method has good generalization performance on other recognition tasks. We adopt the baseline Faster R-CNN detection framework and study only the effect of replacing the VGG-16 backbone with ResNet-101, holding the detection hyperparameters fixed. Table 8 reports PASCAL VOC results: mAP improves from 73.2 to 76.4 on VOC 2007 (trained on 07+12) and from 70.4 to 73.8 on VOC 2012 (trained on 07++12). On COCO (Table 9), the standard mAP@.5 improves from 41.5 to 48.4, and the stricter mAP@[.5, .95] — which rewards tighter localization — improves from 21.2 to 27.2. This 6.0-point absolute gain is a 28% relative improvement, and is comparable to the gain on the looser mAP@.5, indicating that the deeper representations help both recognition and localization. Because only the backbone changed, the improvement is attributable to the learned representation rather than the detection pipeline.

5 Conclusion and Limitations

Conclusion. We have presented a residual learning framework that reformulates the layers of a deep network as learning residual functions with reference to the layer inputs, realized by parameter-free identity shortcut connections. This reformulation eases the optimization of very deep networks: whereas plain networks of increasing depth suffer a degradation in training accuracy, their residual counterparts are readily optimized and gain accuracy from considerably increased depth. The framework requires no change to the optimizer and adds no parameters relative to the plain counterpart, yet it scales to depths far beyond what was previously trainable, and the depth of the resulting representations proves to be of central importance not only for image classification but also for downstream localization-sensitive recognition tasks.

Limitations. Several boundaries of the present study should be stated plainly. First, the claim that the degradation of plain networks stems from optimization difficulty rather than vanishing gradients is argued from batch-normalized signal variances and healthy gradient norms, and is supported by the observation that training longer does not close the gap; it is *not* a formal proof, and the conjecture of exponentially low convergence rates for plain nets remains open. Likewise, the premise that stacked nonlinear layers can asymptotically approximate the residual function is an open question, not an established fact. Second, the per-iteration training trajectories that motivate the “throughout training” phrasing are reported in the original analysis but are not tabulated as per-iteration data in the artifact underlying this manuscript; the figures here therefore present reported endpoint errors rather than redrawn curves. Third, residual learning eases optimization but does not by itself prevent overfitting: at extreme depth on a small dataset, the 1202-layer CIFAR network trains without optimization difficulty yet generalizes worse than a much smaller residual net, and we did not apply strong regularization to counter this. The study scales depth only — it does not investigate width — and uses first-order SGD throughout, with no comparison against second-order or specialized solvers.

Future work. Two concrete directions follow. Combining extremely deep residual nets with explicit regularization such as maxout or dropout would test whether the overfitting seen at 1202 layers on CIFAR-10 can be removed while retaining the clean optimization behavior. Pairing residual learning with stronger regularizers, or with the box-refinement, global-context, and multi-scale testing extensions used in the competition entries, is a natural path to quantify how much of the detection gain can be pushed further on top of the backbone improvement.

References

- [1] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.
- [2] Christopher M. Bishop. *Neural Networks for Pattern Recognition*. Oxford University Press, 1995.
- [3] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (VOC) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.
- [4] Ross Girshick. Fast R-CNN. In *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [5] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014.
- [6] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.
- [7] Ian J. Goodfellow, David Warde-Farley, Mehdi Mirza, Aaron Courville, and Yoshua Bengio. Maxout networks. In *International Conference on Machine Learning (ICML)*, 2013.
- [8] Kaiming He and Jian Sun. Convolutional neural networks at constrained time cost. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. In *European Conference on Computer Vision (ECCV)*, 2014.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [11] Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.
- [12] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [13] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015.
- [14] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 33(1):117–128, 2011.
- [15] Hervé Jégou, Matthijs Douze, Cordelia Schmid, and Patrick Pérez. Aggregating local descriptors into a compact image representation. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2010.
- [16] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio

- Guadarrama, and Trevor Darrell. Caffe: Convolutional architecture for fast feature embedding. In *ACM International Conference on Multimedia (MM)*, 2014.
- [17] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [18] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2012.
- [19] Yann LeCun, Bernhard Boser, John S. Denker, Donnie Henderson, R. E. Howard, Wayne Hubbard, and Lawrence D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551, 1989.
- [20] Yann LeCun, Léon Bottou, Genevieve B. Orr, and Klaus-Robert Müller. Efficient backprop. In *Neural Networks: Tricks of the Trade*. 1998.
- [21] Chen-Yu Lee, Saining Xie, Patrick Gallagher, Zhengyou Zhang, and Zhuowen Tu. Deeply-supervised nets. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2015.
- [22] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. *arXiv preprint arXiv:1312.4400*, 2013.
- [23] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, 2014.
- [24] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [25] Guido Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2014.
- [26] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *International Conference on Machine Learning (ICML)*, 2010.
- [27] Florent Perronnin and Christopher Dance. Fisher kernels on visual vocabularies for image categorization. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2007.
- [28] Tapani Raiko, Harri Valpola, and Yann LeCun. Deep learning made easier by linear transformations in perceptrons. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2012.
- [29] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2015.
- [30] Shaoqing Ren, Kaiming He, Ross Girshick, Xiangyu Zhang, and Jian Sun. Object detection networks on convolutional feature maps. *arXiv preprint arXiv:1504.06066*, 2015.
- [31] Brian D. Ripley. *Pattern Recognition and Neural Networks*. Cambridge University Press, 1996.
- [32] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. FitNets: Hints for thin deep nets. *International Conference on Learning Representations (ICLR)*, 2015.
- [33] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.
- [34] Andrew M. Saxe, James L. McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv preprint arXiv:1312.6120*, 2013.
- [35] Nicol N. Schraudolph. Accelerated gradient descent by factor-centering decomposition. *Technical Report IDSIA-33-98*, 1998.
- [36] Nicol N. Schraudolph. Centering neural network gradient factors. *Neural Networks: Tricks of the Trade*, 1998.
- [37] Pierre Sermanet, David Eigen, Xiang Zhang, Michaël Mathieu, Rob Fergus, and Yann LeCun. OverFeat: Integrated recognition, localization and detection using convolutional networks. *International Conference on Learning Representations (ICLR)*, 2014.
- [38] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2015.
- [39] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015.
- [40] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Training very deep networks. *arXiv preprint arXiv:1507.06228*, 2015.
- [41] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich.

- Going deeper with convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [42] Richard Szeliski. Fast surface interpolation using hierarchical basis functions. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 12(6):513–528, 1990.
- [43] Richard Szeliski. Locally adapted hierarchical basis preconditioning. In *ACM Transactions on Graphics (SIGGRAPH)*, 2006.
- [44] Tommi Vatanen, Tapani Raiko, Harri Valpola, and Yann LeCun. Pushing stochastic gradient towards second-order methods – backpropagation learning with transformations in nonlinearities. In *Advances in Neural Information Processing Systems (NIPS)*, 2013.
- [45] Andrea Vedaldi and Brian Fulkerson. VLFeat: An open and portable library of computer vision algorithms, 2008.
- [46] William N. Venables and Brian D. Ripley. *Modern Applied Statistics with S-PLUS*. Springer, 1999.
- [47] Matthew D. Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European Conference on Computer Vision (ECCV)*, 2014.

A Implementation Details

ImageNet training. Our ImageNet models follow the training recipe established for deep batch-normalized networks. The image is resized with its shorter side sampled in $[256, 480]$ for scale augmentation; a 224×224 crop is randomly sampled from the image or its horizontal flip, with the per-pixel mean subtracted and standard color augmentation applied. Batch normalization is adopted right after each convolution and before activation. Weights are initialized with the MSRA scheme, and all networks are trained from scratch. We use SGD with a mini-batch size of 256, momentum 0.9, and weight decay 10^{-4} . The learning rate starts at 0.1 and is divided by 10 when the validation error plateaus, and the models are trained for up to 60×10^4 iterations. We do not use dropout. In testing, we report 10-crop results following standard practice [18] for the comparison tables; for the best single-model results we adopt fully-convolutional multi-scale testing [38, 37, 24] at scales $\{224, 256, 384, 480, 640\}$ and average the scores. All activations are ReLU [26].

CIFAR-10 training. On CIFAR-10 [17] we use a mini-batch size of 128 on two GPUs, momentum 0.9, and weight decay 10^{-4} , with MSRA initialization and batch normalization and no dropout. Following standard practice [21], we pad the 32×32 image by 4 pixels on each side and sample a random 32×32 crop from the padded

image or its horizontal flip; only the original 32×32 image is used at test time, with per-pixel mean subtraction. The learning rate starts at 0.1 and is divided by 10 at 32,000 and 48,000 iterations, with training terminated at 64,000 iterations on a 45k/5k train/validation split. For the 110-layer model only, we first warm up at a learning rate of 0.01 for ~ 400 iterations until the training error is below $\sim 80\%$, then restore 0.1.

Detection fine-tuning. For detection we use the baseline Faster R-CNN system with 4-step alternating training. The ResNet-101 backbone shares the full-image convolutional features through the stage with total stride 16; region-of-interest pooling is applied before the final convolutional stage, which then acts as the per-region equivalent of the fully-connected layers, with the classification and box regression heads replaced for the target task. Batch-normalization statistics are frozen during fine-tuning to reduce memory consumption, so batch normalization behaves as an affine transform. The detection network is trained on 8 GPUs with a mini-batch of 8 images for the region-proposal step and 16 for the detector step; the learning rate is 0.001 for 240k iterations and then 0.0001 for 80k iterations.

Environment. The original work does not commit to a specific framework in the manuscript; the reference implementation of the era is a standard convolutional training stack [16] with per-pixel mean subtraction and color/scale augmentation data layers, with normalization following established practice [20]. The ImageNet GPU count is not specified, while the mini-batch size of 256 implies a multi-GPU setup; CIFAR-10 uses two GPUs and detection uses eight. Random seeds are not specified; the 110-layer CIFAR model is reported as five runs with $\text{mean} \pm \text{std}$, so averaging over seeds is performed for that depth.

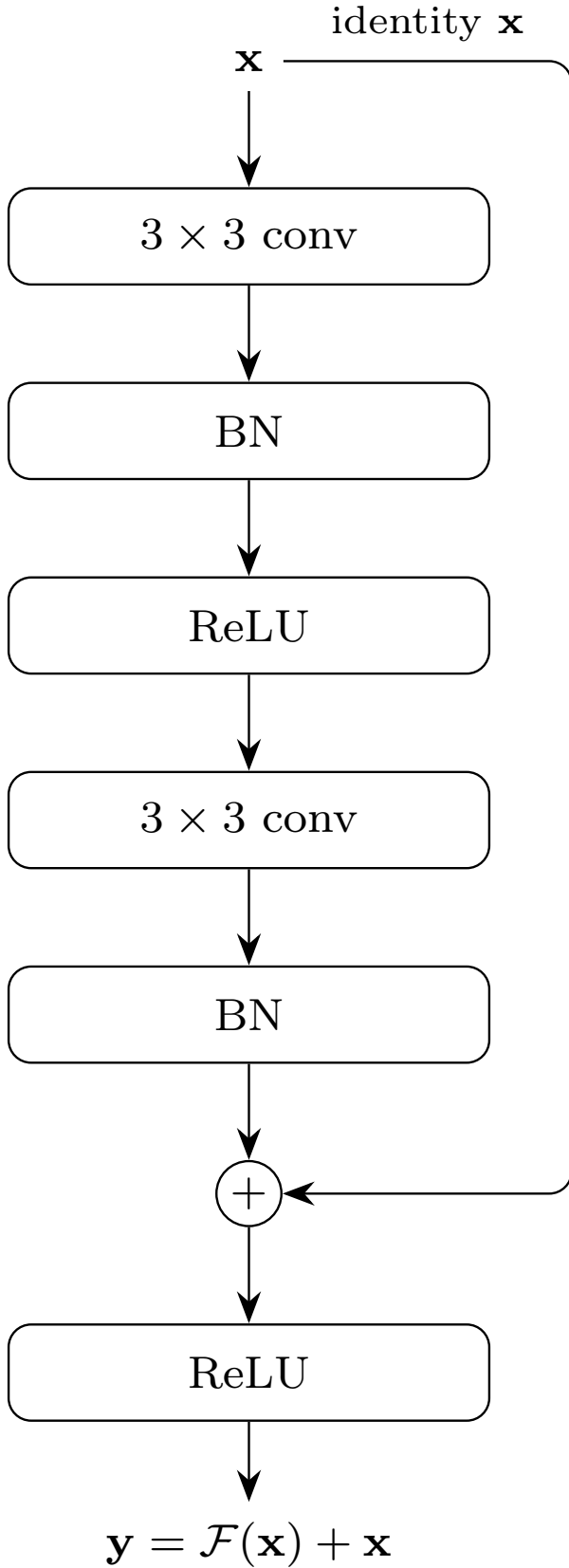


Figure 1: Residual learning: a basic two-layer building block. The shortcut connection performs identity mapping and its output is added element-wise to the output of the stacked layers (Eq. (1)). Identity shortcuts add neither parameters nor computational complexity. *Source: architecture.md (basic block); algorithm.md (Eqn. 1).* 10

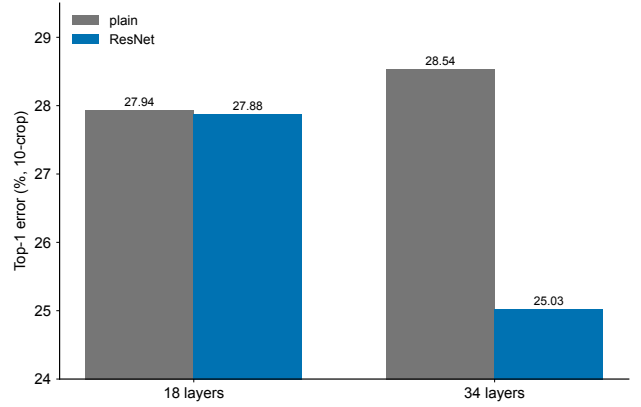


Figure 2: ImageNet validation top-1 error (% , 10-crop) for plain vs. residual nets at 18 and 34 layers. **The 34-layer plain net degrades to 28.54% while the 34-layer ResNet improves to 25.03%.** Bars are the reported *endpoint* validation errors from Table 2; the paper’s per-iteration training curves (Fig. 4) are not tabulated in the source artifact and are therefore not redrawn here. N: single run per configuration (ImageNet val, 10-crop). *Source: table2_imagenet_plain_vs_residual.md.*

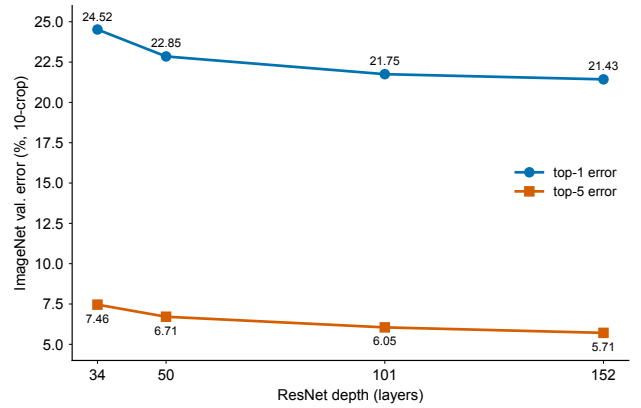


Figure 3: ImageNet validation error (% , 10-crop) of residual nets vs. depth. **Top-1 error falls monotonically from 24.52% at 34 layers to 21.43% at 152 layers.** Markers are reported endpoint errors from Table 3 (ResNet-34 shown as option B). N: single run per depth (ImageNet val, 10-crop). *Source: table3_imagenet_validation_full.md.*

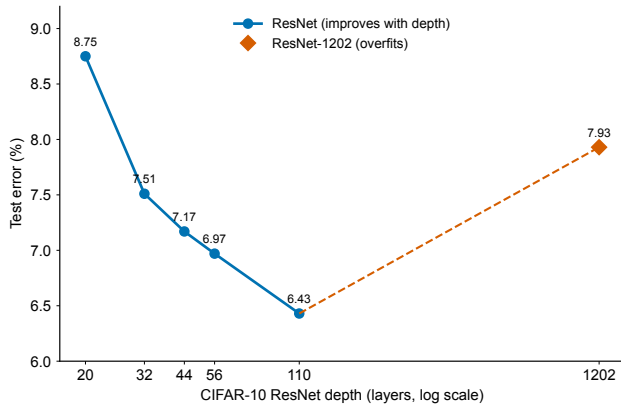


Figure 4: CIFAR-10 test error vs. ResNet depth. **Test error improves from 8.75% at 20 layers to 6.43% at 110 layers, then rises to 7.93% at 1202 layers due to overfitting.** Markers are reported endpoint test errors from Table 6 (the 110-layer point is the 5-run best). N: 5 runs for ResNet-110, single run otherwise. *Source: table6_cifar10.md.*